

Lesson 5: Async/Await and Working With APIs

Mission Brief

Chaining `.then()` after `.then()` works, but as your ground station requests get more complex — search a city, then fetch its forecast, then fetch its alerts — those chains turn into a tangled mess of nested callbacks. Today Lumexa Mission Control upgrades to `async / await`, a cleaner way to write asynchronous code that reads almost like it runs step-by-step, even though it's still fully asynchronous underneath.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what `async` and `await` do and how they relate to Promises
- Convert a `.then()` chain into an `async` function using `await`
- Handle errors in async code using `try / catch`
- Chain multiple dependent API calls (like geocoding a city, then fetching its weather)
- Show and hide loading states correctly around asynchronous operations

What You'll Build

A "City Weather Lookup" tool that takes a typed city name, uses Open-Meteo's **geocoding API** to convert it into coordinates, then uses those coordinates to fetch real current weather — two chained, real API calls — all written with `async / await` and `try / catch`, with a loading indicator and real error messages for invalid cities or network failures.

Prerequisites

- Lesson 4 (Fetch API, Promises, `.then()`)
- Lesson 3 (event listeners, for wiring up a search button)

Key Concepts

- `async function` declarations
- The `await` keyword
- `try / catch / finally`
- Chaining dependent asynchronous calls
- Loading states and disabling UI during a request
- Geocoding (turning a place name into latitude/longitude)

Concept Explanation

`async / await` is not a different technology from Promises — it's **syntax sugar** built directly on top of them, designed to make asynchronous code look and read like ordinary, synchronous, top-to-bottom code. Marking a function `async` (`async function getWeather() { }`) does two things: it makes that function always return a Promise, and it unlocks the ability to use the `await` keyword inside it.

`await` can only be used inside an `async` function. Placing `await` before a Promise-returning expression (like `fetch(url)`) pauses that function's execution — without freezing the rest of the page — until the Promise settles, then "unwraps" the result directly into a normal variable. Compare:

```
// .then() version
fetch(url).then((response) => response.json()).then((data) => console.log(data));

// async/await version — same behavior, reads top-to-bottom
async function loadData() {
  const response = await fetch(url);
  const data = await response.json();
  console.log(data);
}
```

Both versions do exactly the same thing under the hood; `async / await` just removes the nested callback structure, which becomes especially valuable once you need to make a **second** API call that depends on the result of the first — like this lesson's geocode-then-forecast pattern. Written with `.then()` , that would require nesting one chain inside another; written with `await` , it's just two lines in a row.

Error handling changes shape too: instead of a `.catch()` at the end of a chain, `async / await` uses ordinary `try { } catch (error) { }` blocks, exactly like error handling in non-async code. Anything that throws inside the `try` block — a network failure, a manually thrown `Error` , or a bug — is caught by the matching `catch` block. An optional `finally { }` block runs regardless of success or failure, which is the perfect place to turn off a loading spinner or re-enable a disabled button.

A realistic multi-step API workflow — like this lesson's — often looks like: (1) validate user input, (2) call one API to resolve a name into structured data (geocoding a city name into coordinates), (3) use that result to call a second API (fetching weather for those coordinates), (4) update the UI, and (5) handle failure at *any* of those steps gracefully. `async / await` with `try / catch` is the cleanest way to express that whole sequence.

Guided Coding

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>City Weather Lookup</title>
  <style>
    body { font-family: sans-serif; background: #0b0e1a; color: #e6ecff; padding: 2rem; max-
width: 480px; }
    #result { margin-top: 1rem; min-height: 3em; }
    .error { color: #f87171; }
    .loading { color: #93c5fd; }
    button:disabled { opacity: 0.5; cursor: not-allowed; }
  </style>
</head>
<body>
  <h1>City Weather Lookup</h1>
  <input type="text" id="city-input" placeholder="Enter a city name..." />
  <button id="search-btn">Search</button>
  <div id="result">Search for a city to see live weather.</div>
  <script src="script.js"></script>
</body>
</html>
```

```

// script.js

const cityInput = document.querySelector("#city-input");
const searchButton = document.querySelector("#search-btn");
const resultDiv = document.querySelector("#result");

// STEP 1: geocode a city name into latitude/longitude (Open-Meteo, no API key)
async function geocodeCity(cityName) {
  const url = `https://geocoding-api.open-meteo.com/v1/search?
name=${encodeURIComponent(cityName)}&count=1&language=en&format=json`;

  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`Geocoding service error (status ${response.status})`);
  }

  const data = await response.json();

  // Open-Meteo returns { results: [...] } – or no "results" key at all if nothing matched.
  if (!data.results || data.results.length === 0) {
    throw new Error(`No city found matching "${cityName}"`);
  }

  const firstMatch = data.results[0];
  return {
    name: firstMatch.name,
    country: firstMatch.country,
    latitude: firstMatch.latitude,
    longitude: firstMatch.longitude,
  };
}

// STEP 2: fetch current weather for known coordinates
async function fetchCurrentWeather(latitude, longitude) {
  const url =
    `https://api.open-meteo.com/v1/forecast?latitude=${latitude}&longitude=${longitude}` +
    `&current=temperature_2m,relative_humidity_2m,weather_code&timezone=auto`;

  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`Forecast service error (status ${response.status})`);
  }

  return response.json();
}

// STEP 3: orchestrate both calls, with loading state and error handling
async function searchWeather() {

```

```

const cityName = cityInput.value.trim();

if (cityName === "") {
  resultDiv.textContent = "Please enter a city name.";
  resultDiv.classList.add("error");
  return;
}

searchButton.disabled = true;
resultDiv.classList.remove("error");
resultDiv.classList.add("loading");
resultDiv.textContent = `Searching for "${cityName}"...`;

try {
  const location = await geocodeCity(cityName);
  const weatherData = await fetchCurrentWeather(location.latitude, location.longitude);
  const temperature = weatherData.current.temperature_2m;
  const humidity = weatherData.current.relative_humidity_2m;

  resultDiv.classList.remove("loading");
  resultDiv.textContent =
    `${location.name}, ${location.country}: ${temperature}°C, ${humidity}% humidity`;
} catch (error) {
  resultDiv.classList.remove("loading");
  resultDiv.classList.add("error");
  resultDiv.textContent = `Could not load weather: ${error.message}`;
  console.error(error);
} finally {
  // Runs whether the search succeeded or failed — always re-enable the button.
  searchButton.disabled = false;
}
}

searchButton.addEventListener("click", searchWeather);
cityInput.addEventListener("keydown", (event) => {
  if (event.key === "Enter") {
    searchWeather();
  }
});

```

Code Walkthrough

- **geocodeCity**: an `async` function that `await`s the geocoding fetch, checks `.ok`, parses JSON, and — critically — checks whether `data.results` exists and has at least one entry, since Open-Meteo's geocoding API simply omits the `results` key entirely when no city matches, rather than returning an empty error object. `encodeURIComponent` safely escapes the city name for use in a URL (handling spaces, accents, etc.).

- `fetchCurrentWeather` : a second, independent `async` function, kept separate from geocoding so each function has one clear responsibility (a good software design habit).
- `searchWeather` : the orchestrator. Notice `await geocodeCity(cityName)` and `await fetchCurrentWeather(...)` sit on consecutive lines, reading like ordinary sequential code, even though real network round-trips happen at each `await`.
- **Loading state**: before the `try` block runs, the button is disabled and a "Searching..." message with a `loading` class appears; this happens *synchronously*, immediately on click, before any `await` pauses execution.
- **try / catch** : any thrown `Error` — from either helper function, or an unexpected network failure — is caught in one place, and a friendly message is shown.
- **finally** : guarantees `searchButton.disabled = false` runs whether the search succeeded, failed with a message, or threw an unexpected error, so the button is never left stuck in a disabled state.

Student Mission

1. Add a small "feels like" temperature to the display by requesting `apparent_temperature` in the `current` parameter and showing it alongside the main temperature.
2. Add input validation: if the user searches for the exact same city twice in a row, skip the network calls and show a message "Already showing weather for " instead (you'll need a variable to remember the last successful city).
3. Convert one of the two helper functions back into `.then()` syntax as an exercise, then convert it back to `async / await` — write a comment explaining which version you find clearer and why.

Challenge

Extend `geocodeCity` so that if the geocoding API returns **multiple** matches for ambiguous city names (e.g., "Springfield"), instead of blindly picking `data.results[0]`, you display all matches (name, country, admin1/region) as a small clickable list, and only call `fetchCurrentWeather` once the user picks one. This requires restructuring `searchWeather` to `await` a user click before continuing — a realistic multi-step async UI challenge.

Experiment/Extension

Temporarily add `await new Promise((resolve) => setTimeout(resolve, 3000));` as the first line inside the `try` block in `searchWeather`, to artificially slow the request down. Confirm your loading state is visible for those 3 seconds, then remove the line. This is a common real-world technique for testing loading states without needing an actually slow network.

Common Mistakes

- Forgetting the `async` keyword on a function that uses `await` inside it — a `SyntaxError: await is only valid in async functions`.
- Wrapping `await` calls in `try / catch` but forgetting the `finally` block, leaving a button permanently disabled after an error.
- Not `await`-ing `response.json()` (writing `const data = response.json();` without `await`) and then trying to use `data` as if it were already the parsed object, when it's actually still a pending Promise.
- Assuming a missing key (like `data.results` being `undefined`) will throw automatically — it doesn't; you must check for it explicitly, as shown in `geocodeCity`.
- Running two independent, unrelated `await` calls one after another when they could run in parallel with `Promise.all()` (an intermediate-level optimization, worth knowing exists even if not required here).

Debugging Tips

- Add `console.log("about to fetch weather for", location)` right before the second `await` call to confirm the first step actually produced valid coordinates before moving on.
- Use the debugger: put a `debugger;` statement inside `searchweather`'s `try` block, open DevTools, and step through line-by-line with the step-over button, watching each variable populate as each `await` resolves.
- If you see `Uncaught (in promise) TypeError`, check the **Console**'s expandable stack trace — it names the exact function and line, and clicking it jumps to that source line.
- Check the **Network tab** to confirm two separate requests fire in sequence (geocoding, then forecast) — the forecast request's URL should contain the exact coordinates returned by the geocoding response.

Check Your Understanding

1. What relationship does `async / await` have with Promises?
2. Why must `await` only be used inside a function marked `async`?
3. How does error handling in `async / await` code differ syntactically from `.then()` chains?
4. Why is `finally` a good place to reset a loading state or re-enable a button?
5. Why does `geocodeCity` check `data.results.length === 0` instead of assuming a match always exists?

Mini Quiz + Answer Key

- Q1.** What does an `async` function always return? A) `undefined` B) A Promise C) A string D) The awaited value directly, synchronously
- Q2.** Where should you catch errors thrown by an `await` ed call? A) In a `.then()` B) In a `try / catch` block C) You cannot catch them D) In the function's return statement
- Q3.** What does a `finally` block guarantee? A) It only runs on success B) It only runs on failure C) It runs regardless of success or failure D) It replaces `catch`

Q4. Why does `searchWeather` disable the button before the `try` block, not inside it? A) It doesn't matter where B) So the button is disabled immediately and synchronously, before any `await` pauses execution C) `try` blocks cannot contain UI code D) To avoid a syntax error

Answer Key: 1-B, 2-B, 3-C, 4-B

Lesson Recap

You learned that `async / await` is a cleaner syntax layered on top of Promises, how to chain multiple dependent API calls (geocode a city, then fetch its weather) using sequential `await` statements, how to handle errors with `try / catch / finally`, and how to manage loading states correctly around real asynchronous work.

Homework

Extend the City Weather Lookup with a "Use My Location" button that uses the browser's `navigator.geolocation.getCurrentPosition()` (which itself is callback-based — wrap it in a `new Promise()` so you can `await` it) to get the user's real latitude/longitude, then call `fetchCurrentWeather` directly, skipping the geocoding step entirely.

Portfolio Project Connection

This lesson is the async engine of **Project 05, the Live Weather Dashboard**: the geocode-then-forecast pattern, loading states, and `try / catch` error handling you practiced here are used directly (and extended with a multi-day forecast) in that project's real implementation.